Trappole d'esame — Dominio 4: Prompt engineering e output strutturato

> **Come usare questo file.** Per ogni trappola: risposta sbagliata (❌), perché, risposta corretta (✅), segnale nella domanda (🔎). ⚠️ **Guida vs docs** segnala dove la guida d'esame e la documentazione attuale (ottobre 2026) divergono: all'esame vale la guida.

Regole d'oro del dominio

1. **Criteri espliciti e categorici** battono istruzioni vaghe ("sii conservativo") e soglie di confidenza.
2. **La confidenza auto-dichiarata del modello non è calibrata**: usala solo dopo averla calibrata su dati etichettati.
3. **Formato incoerente → esempi (few-shot), non altre istruzioni.** 2-4 esempi mirati, con il ragionamento.
4. **Lo schema JSON elimina gli errori di sintassi, non quelli di significato** → serve validazione semantica (es. Pydantic).
5. **Retry solo con l'errore specifico come feedback**, e solo se l'informazione esiste nella fonte.
6. **Batch API = economica ma fino a 24 h, senza SLA** → solo per lavori che possono aspettare.
7. **Revisione con istanza indipendente e passaggi per file + integrazione.**

---

1. System prompt con criteri espliciti

Trappola 1 — "Sii conservativo" o "segnala solo risultati ad alta confidenza" come miglioramento del prompt

Perché è sbagliato: il modello non ha un'interpretazione operativa di "conservativo"; la precisione non migliora.

✅ **Criteri categorici specifici** su cosa segnalare e cosa ignorare. Esempio:

- segnala: SQL costruito concatenando input utente, segreti hardcoded;
- ignora: stile, naming, TODO.

Trappola 2 — Pensare che una soglia di confidenza risolva i falsi positivi

Perché è sbagliato: la confidenza dichiarata da un LLM è mal calibrata.

✅ Criteri espliciti con **esempi di codice concreti**. Il routing per confidenza può essere utile, ma **solo dopo** aver definito i criteri (e calibrato le soglie, vedi trappola 20).

Trappola 3 — Lasciare attive tutte le categorie di review mentre si sistema quella con molti falsi positivi

Perché è sbagliato: molti falsi positivi in una categoria fanno perdere fiducia in **tutte** le categorie: gli sviluppatori iniziano a ignorare ogni commento.

✅ **Disattiva temporaneamente** la categoria problematica, migliorane il prompt, poi riattivala.

---

2. Few-shot prompting

Trappola 4 — "Aggiungi istruzioni più dettagliate" quando il formato dell'output è incoerente

Perché è sbagliato: le istruzioni *descrivono* il formato; gli esempi lo *mostrano*.

✅ Pochi **esempi del formato esatto** sono la correzione più efficace.

Trappola 5 — Molti esempi per coprire ogni caso possibile

Perché è sbagliato: si sprecano token senza beneficio proporzionale. Ciò che insegna al modello a generalizzare è il **ragionamento** dentro ogni esempio, non il numero di esempi.

✅ Guida d'esame: **2-4 esempi mirati con il ragionamento**, scelti sui casi ambigui o di confine.

⚠️ **Guida vs docs:** la documentazione attuale suggerisce 3-5 esempi. All'esame scegli l'opzione con "pochi esempi mirati".

Trappola 6 — Soglie di confidenza per decisioni di giudizio incoerenti

Perché è sbagliato: la confidenza non è calibrata e non affronta la causa: il modello non sa come decidere nei casi ambigui.

✅ Esempi few-shot che mostrano **la decisione corretta sui casi ambigui**, con motivazione.

---

3. Output strutturato tramite tool use

Trappola 7 — Credere che `tool_use` con schema JSON prevenga tutti gli errori di estrazione

Perché è sbagliato: lo schema garantisce la **forma**, non la **correttezza**. Restano gli errori semantici:

- voci che non sommano al totale dichiarato;
- valori nel campo sbagliato;
- valori inventati.

✅ Valida separatamente la semantica (vedi trappola 13).

⚠️ **Guida vs docs:** la documentazione attuale garantisce input conformi allo schema solo con `strict: true`. All'esame: lo schema elimina gli errori sintattici, non quelli semantici.

Trappola 8 — Affidarsi a `tool_choice: "auto"` per garantire una chiamata allo strumento

Perché è sbagliato: con `auto` il modello può rispondere in testo e saltare lo strumento.

✅ Guida d'esame: più schemi possibili e tipo di documento sconosciuto → `tool_choice: "any"` (il modello deve chiamare uno degli strumenti, scegliendo quello adatto). Schema unico noto → strumento forzato.

⚠️ **Guida vs docs:** la documentazione attuale restituisce errore 400 su `any` per i modelli più recenti (Opus 5.5, Sonnet 5.5, Fable 5.1, Mythos 5.1) e indica `auto` con `strict: true`. All'esame rispondi secondo la guida.

Trappola 9 — Rendere obbligatori tutti i campi dello schema "per completezza"

Perché è sbagliato: un campo obbligatorio spinge il modello a **inventare** un valore quando la fonte non lo contiene.

✅ Campi che un documento potrebbe non avere → **nullable** (`"type": ["string", "null"]`), così il modello può restituire `null`. Restano obbligatori solo i campi sempre presenti.

---

4. Validazione, retry e feedback loop

```
estrai → valida (schema + regole semantiche)
   ├─ ok → accetta
   ├─ errore correggibile → retry CON il messaggio d'errore specifico
   ├─ informazione assente nella fonte → null o revisione umana
   └─ fonte contraddittoria → estrai entrambi i valori + flag → revisione
```

Trappola 10 — Pensare che il retry risolva sempre i fallimenti di estrazione

Perché è sbagliato: il retry corregge errori di formato, struttura e valori fuori posto, ma **non può far apparire informazioni che la fonte non contiene**.

✅ Retry per gli errori risolvibili (con l'errore di validazione come feedback); informazione assente → `null` se lo schema lo consente, altrimenti **revisione umana**.

Trappola 11 — Retry senza includere l'errore di validazione specifico

Perché è sbagliato: senza feedback il modello ripete lo stesso errore.

✅ Rimanda l'errore preciso, es. "*Le voci sommano a 450 £ ma il totale dichiarato è 500 £*", così il modello può correggersi.

Trappola 12 — Riprovare, o scegliere un valore, quando il documento sorgente si contraddice

Perché è sbagliato: il retry non può risolvere un'incoerenza che sta nella fonte; scegliere un valore è tirare a indovinare.

✅ Estrai **entrambi** i valori (`calculated_total`, `stated_total`) oppure imposta un flag `conflict_detected`, poi manda il caso a **revisione**.

Trappola 13 — Considerare Pydantic ridondante perché `tool_use` impone già uno schema JSON

Perché è sbagliato: lo schema JSON non esprime regole **tra campi** (somme che devono tornare, date in ordine, coerenze).

✅ I validatori Pydantic codificano queste regole e producono i **messaggi di errore specifici per campo** che il retry loop rimanda al modello.

---

5. Elaborazione batch (Message Batches API)

	API sincrona	Batch API
Latenza	immediata	fino a 24 h, **senza SLA**
Costo	pieno	circa 50% in meno
Adatta a	pre-merge, chat, review in tempo reale	report notturni, analisi massive
Strumenti client a metà richiesta	✅	❌

Trappola 14 — Spostare tutti i workflow sul batch per risparmiare

Perché è sbagliato: i workflow in cui qualcuno aspetta il risultato (check pre-merge, review in tempo reale) non tollerano fino a 24 ore.

✅ Batch **solo per i workflow che tollerano la latenza**.

Trappola 15 — Contare sul fatto che i risultati batch "di solito arrivano presto"

Perché è sbagliato: spesso arrivano prima, ma non c'è garanzia. Un workflow bloccante non si può basare sul caso migliore.

✅ Progetta considerando il **massimo di 24 ore**.

Trappola 16 — Usare il batch quando un tuo strumento deve girare a metà richiesta e la richiesta deve continuare col risultato

Perché è sbagliato: una richiesta batch non può fermarsi, aspettare che tu esegua lo strumento e poi riprendere.

✅ Esegui quel passaggio in modo **sincrono**, oppure risolvi la chiamata allo strumento **prima** dell'invio o **tra un batch e l'altro**.

⚠️ **Guida vs docs:** oggi il batch esegue gli strumenti *lato server*, ma gli strumenti *tuoi* (client) fanno terminare la richiesta con `tool_use`. La conclusione per l'esame non cambia.

---

6. Revisione multi-istanza e multi-passaggio

Trappola 17 — Auto-revisione nella stessa sessione

Perché è sbagliato: il modello conserva il ragionamento con cui ha generato l'output ed è poco propenso a contraddirsi.

✅ Un'**istanza indipendente** senza quel contesto trova molti più problemi sottili. (Stesso principio di D3, trappola 20.)

Trappola 18 — Un unico passaggio per una review su molti file

Perché è sbagliato: la diluizione dell'attenzione produce profondità disomogenea, bug mancati e commenti contraddittori.

✅ **Un passaggio locale per file** + **un passaggio di integrazione cross-file**.

Trappola 19 — Passare a un modello con finestra di contesto più grande per la diluizione dell'attenzione

Perché è sbagliato: la finestra più grande contiene più testo, ma l'attenzione resta disomogenea tra i file.

✅ Passaggi focalizzati per file (vedi trappola 18).

🔎 "Finestra più grande", "modello più potente", "prompt più lungo" come soluzione alla diluizione sono quasi sempre distrattori (anche in D1 e D5).

Trappola 20 — Instradare automaticamente le revisioni con punteggi di confidenza non calibrati

Perché è sbagliato: la confidenza grezza auto-dichiarata è mal calibrata.

✅ **Calibra le soglie su un set di validazione etichettato** prima di usarle per decidere il routing.